

# Backend — ingest, arbiter, fan-out

**Owner:** *TBD*

**Depends on:** `docs/PROTOCOL.md`

**Deliverable for M1:** a deployed API serving the simulated feeder over REST + WebSocket.

You own `decide()`. It is the single most reused piece of code in the project — the gateway will run a C port of your logic, and the firmware's local detector must agree with it. Write it as a pure function and test it hard.

---

## 1. Stack

| Concern   | Demo                                            | Production               |
|-----------|-------------------------------------------------|--------------------------|
| Framework | FastAPI + Uvicorn                               | same                     |
| Ingest    | in-process pub/sub from <code>simulator/</code> | MQTT (EMQX or Mosquitto) |

| Concern | Demo                       | Production                                   |
|---------|----------------------------|----------------------------------------------|
| Store   | in-memory ring buffer      | TimescaleDB / Postgres                       |
| Fan-out | native WebSocket           | same, behind Redis pub/sub if multi-instance |
| Deploy  | Render / Railway free tier | containerised, wherever KSEBL permits        |

Skip the database for M1. A ring buffer of the last 30 minutes is enough for the demo and saves you a day.

## 2. Setup

```
cd backend
python -m venv .venv && source
.venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

The simulator starts as a FastAPI lifespan background task, so a single command brings up the whole demo backend. This matters: the fewer moving parts on demo day, the better.

### 3. Structure

```
backend/app/
├─ main.py           FastAPI app,
lifespan, CORS, router mounting
├─ arbiter.py        decide() – pure, no
I/O, THE core file
├─ models.py         Pydantic models
mirroring PROTOCOL.md exactly
├─ state.py          in-memory feeder
state, ring buffers, node registry
├─ ingest.py         telemetry intake;
swappable simulator ↔ MQTT
├─ events.py         event construction,
trail assembly, latency computation
├─ ws.py             WebSocket connection
manager, broadcast, heartbeat
├─ routes/
|   ├─ feeders.py
|   ├─ nodes.py
|   ├─ events.py
|   └─ simulate.py   scenario trigger
endpoints (demo only)
└─ config.py         ArbiterConfig
loading, per-feeder overrides
tests/
├─ test_arbiter.py   runs every vector in
docs/vectors/
└─ test_ingest.py
```

### 4. API surface

| Method | Path                               | Returns                                                          |
|--------|------------------------------------|------------------------------------------------------------------|
| GET    | /feeders                           | feeder list with<br>polyline geometry<br>and node<br>coordinates |
| GET    | /feeders/{id}                      | one feeder,<br>current mode,<br>ArbiterConfig                    |
| GET    | /nodes?feeder_id=                  | current state of<br>every node                                   |
| GET    | /nodes/{id}/history?<br>window=300 | telemetry<br>samples for<br>sparklines                           |
| GET    | /events?<br>feeder_id=&limit=50    | event log, newest<br>first                                       |
| POST   | /events/{id}/ack                   | crew<br>acknowledgement                                          |
| POST   | /simulate/{scenario}               | trigger a scenario<br>— <b>demo build<br/>only</b>               |
| POST   | /feeders/{id}/mode                 | AUTO ↔<br>ALERT_ONLY                                             |

| Method | Path               | Returns                                 |
|--------|--------------------|-----------------------------------------|
| WS     | /stream?feeder_id= | live telemetry, votes, commands, events |
| GET    | /healthz           | liveness, used to warm cold starts      |

WebSocket frames are envelope-wrapped:

```
{ "kind": "telemetry" | "vote" | "command"
  | "event" | "hello", "payload": { ... } }
```

Send a `hello` frame on connect containing the full current state, so the frontend does not need a separate REST call to bootstrap.

## 5. `decide()` — implementation notes

Read section 5 of `docs/PROTOCOL.md` for the signature and veto rules. Beyond that:

- **Pure means pure.** No `time.time()` inside — `now_ms` is a parameter. No logging, no DB reads. This is what makes the test vectors deterministic and what allows a faithful C port.
- **Compute the span, do not guess it.** Walk the ordered node list, find the boundary where upstream is NORMAL and downstream is

SUSPECT. If there is more than one such boundary, return `ALERT`, not `ISOLATE` — ambiguity is not a licence to trip.

- **Latency is measured from the earliest SUSPECT in the trail**, not from when your function ran.
- **Confidence** should fall as vote count approaches the minimum quorum and as vote timestamps approach the window edge. The frontend displays it; a bare boolean looks naive to an engineer on the panel.

## 6. Task list

### Phase 1 — contract in code (day 1)

- [ ] `models.py` — Pydantic models for telemetry, vote, command, event; field names identical to `PROTOCOL.md`
- [ ] `docs/vectors/` fixtures written and committed
- [ ] `test_arbiter.py` written against the vectors, all failing (red first)

### Phase 2 — arbiter (days 2–3)

- [ ] `decide()` implemented, all vectors green
- [ ] All five veto rules implemented and individually tested
- [ ] Property test: no input where every node is SUSPECT ever returns `ISOLATE`

### Phase 3 — service (days 3–5)

- [ ] `state.py` ring buffers, 60 samples per node, OFFLINE detection at 30 s
- [ ] `ingest.py` consuming the simulator's async queue
- [ ] Arbiter invoked on every state transition, not on a polling loop
- [ ] `events.py` builds the trail and computes `latency_ms`
- [ ] `ws.py` broadcast with per-connection queues; a slow client must not stall the loop
- [ ] All REST routes
- [ ] CORS allowing the Vercel origin
- [ ] Deployed, `/healthz` green

### Phase 4 — production hardening (post-M1)

- [ ] MQTT ingest path behind the same `ingest` interface — simulator and broker interchangeable
- [ ] TimescaleDB, 90-day telemetry retention, hypertable on `ts`
- [ ] mTLS per-device certificates; device registry and provisioning flow
- [ ] Alerting: MSG91/Twilio SMS + FCM push, escalation ladder with ack timeout
- [ ] Multi-tenant auth scoped to KSEBL section/division

- [ ] OTA artifact server with signed firmware images
- [ ] IEC 60870-5-104 adapter — even a read-only stub demonstrates you know utilities will not adopt an island system
- [ ] Structured audit log: every mode change and manual command, with actor

## 7. Acceptance criteria for M1

1. `pytest` green, every vector in `docs/vectors/` passing.
2. A break scenario produces an event with `latency_ms < 2000` and a correct `fault_span`.
3. Rain, vegetation, substation-outage and node-offline scenarios all produce **no** `ISOLATE`.
4. WebSocket survives a client disconnect and a simulator restart without a process restart.
5. Cold start to first `hello` frame under 5 s.

## 8. Gotchas

- **The cloud does not trip anything.** Your `/simulate` and arbiter output are for display and audit. In production this same `decide()` runs on the gateway; the cloud copy is a shadow that must agree, not a controller. Say this explicitly if a judge asks where the decision lives.

- **Do not let the arbiter run on a timer.** Event-driven only, or your `latency_ms` inherits your poll interval and stops being an honest number.
- **Free-tier instances sleep.** Add a cron ping or accept the cold start and make the frontend show it gracefully.
- `seq` **resets to 0 on node reboot.** Handle the wrap; do not treat it as a replay attack.
- Keep `/simulate` routes behind an env flag so they are trivially disabled in a production build.